

Contenedores Docker

Imágenes, Dockerfile, multi-stage build y registros

Ingeniería Web Avanzada

Pontificia Universidad Católica de Valparaíso

Segundo semestre 2026

Continuidad con la clase anterior

Ya trabajamos

- Backend y frontend como componentes separados.
- Pipeline CI con `npm ci`, `npm test` y `npm run build`.
- Jobs, steps, runners, caché y artefactos.
- Ejemplo con Angular como frontend y NestJS como backend.

Pregunta para esta clase

Si el código ya se instala, prueba y construye correctamente, ¿cómo lo empaquetamos para ejecutarlo de forma reproducible en distintos ambientes?

Problema: “funciona en mi máquina”

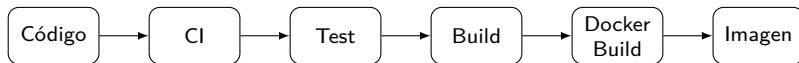
Situación

Un estudiante ejecuta correctamente el backend NestJS en su computador, pero en otro equipo aparecen errores:

- versión distinta de Node.js;
- dependencias instaladas de forma diferente;
- variables de entorno faltantes;
- comandos no documentados;
- diferencias entre sistema operativo local y servidor.

Idea

Docker busca reducir esas diferencias empaquetando la aplicación junto con el entorno necesario para ejecutarla.



Conexión

En CI verificamos que el proyecto se instala, prueba y construye. Con Docker generamos una imagen que contiene lo necesario para ejecutar la aplicación.

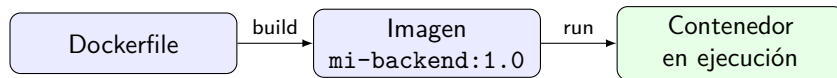
Conceptos principales

Concepto	Descripción
Imagen	Paquete inmutable que contiene aplicación, dependencias, runtime y configuración necesaria para crear contenedores.
Contenedor	Instancia en ejecución de una imagen.
Dockerfile	Archivo de texto con instrucciones para construir una imagen.
Registry	Servicio donde se almacenan y comparten imágenes.
Tag	Etiqueta utilizada para versionar o identificar una imagen.

Regla rápida

Dockerfile construye una **imagen**; una imagen crea un **contenedor**; las imágenes se pueden publicar en un **registro**.

Imagen vs contenedor



Analogía

- La imagen es como una plantilla o paquete preparado.
- El contenedor es la ejecución concreta de esa plantilla.
- Puedo crear muchos contenedores desde la misma imagen.

Primeros comandos Docker

Construir una imagen

```
docker build -t mi-backend:1.0 .
```

Ejecutar un contenedor

```
docker run -p 3000:3000 mi-backend:1.0
```

Ver imágenes y contenedores

```
docker images  
docker ps  
docker ps -a
```

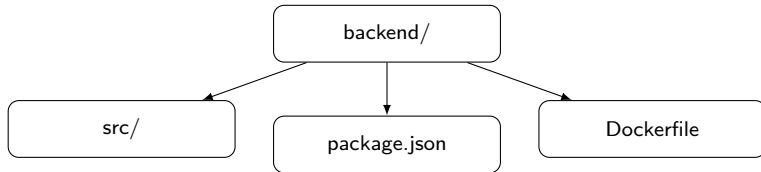
Importante

El punto final en `docker build ... .` indica el contexto de construcción.

¿Qué es el contexto de construcción?

Build context

Es la carpeta que Docker envía al proceso de construcción para que el Dockerfile pueda copiar archivos.



Ejemplo

Si ejecutamos `docker build -t backend .` dentro de `backend/`, el contexto será esa carpeta.

Dockerfile: instrucciones frecuentes

Instrucción	Uso
FROM	Define la imagen base.
WORKDIR	Define el directorio de trabajo.
COPY	Copia archivos al sistema de archivos de la imagen.
RUN	Ejecuta comandos durante la construcción.
EXPOSE	Documenta el puerto usado por la aplicación.
CMD	Define el comando por defecto al iniciar el contenedor.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["npm", "run", "start:dev"]
```

Capas de una imagen

Idea

Cada instrucción del Dockerfile puede generar una capa. Si una capa no cambia, Docker puede reutilizarla en builds posteriores.

CMD npm run start
COPY . .
RUN npm ci
COPY package*.json ./
WORKDIR /app
FROM node:20-alpine

Buena práctica

Copiar primero `package.json` y `package-lock.json` permite reutilizar la capa de dependencias cuando sólo cambia el código.

.dockerignore

¿Para qué sirve?

Evita enviar archivos innecesarios al contexto de construcción. Ayuda a reducir tamaño, tiempo de build y riesgos de exposición.

```
node_modules
dist
.git
.env
coverage
*.log
```

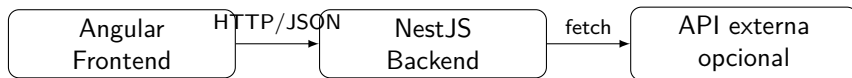
Regla de seguridad

No se deben copiar secretos, archivos `.env` reales ni credenciales dentro de una imagen Docker.

Caso de estudio de la clase

Sistema trabajado

Tenemos un frontend Angular y un backend NestJS. En esta sesión nos enfocaremos en empaquetar el backend que expone un endpoint de productos.



Objetivo técnico

Construir una imagen Docker para ejecutar `GET /productos` desde un contenedor.

Dockerfile básico para NestJS

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

EXPOSE 3000

CMD ["npm", "run", "start:dev"]
```

Uso

Este Dockerfile es útil para comprender el flujo básico. No es necesariamente la mejor opción para producción.

Construir y ejecutar NestJS

Desde la carpeta backend/

```
docker build -t productos-backend:dev .
```

Ejecutar el contenedor

```
docker run -p 3000:3000 productos-backend:dev
```

Probar

```
curl http://localhost:3000/productos
```

Lectura

-p 3000:3000 publica el puerto del contenedor en el puerto 3000 del computador local.

¿Por qué no basta con el Dockerfile básico?

Problemas posibles

- La imagen final puede incluir dependencias de desarrollo.
- Puede contener código fuente innecesario.
- El tamaño puede crecer sin necesidad.
- Se aumenta la superficie de ataque.
- No separa claramente build y runtime.

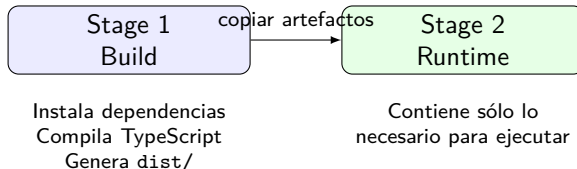
Siguiente paso

Utilizar una construcción **multi-stage**: una etapa construye la aplicación y otra ejecuta sólo lo necesario.

Construcción multi-stage

Idea

Un Dockerfile multi-stage utiliza varias instrucciones FROM. Cada etapa cumple un propósito diferente.



Dockerfile multi-stage para NestJS

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM node:20-alpine AS runtime

WORKDIR /app

ENV NODE_ENV=production

COPY package*.json ./
RUN npm ci --omit=dev

COPY --from=build /app/dist ./dist

EXPOSE 3000
```

Interpretación del Dockerfile multi-stage

Etapa	Responsabilidad
build	Instala dependencias, copia el código y ejecuta <code>npm run build</code> .
runtime	Instala dependencias de producción y copia sólo <code>dist/</code> desde la etapa anterior.
COPY --from=build	Permite traer a la imagen final únicamente el resultado del proceso de construcción.

Resultado esperado

La imagen final queda más limpia y orientada a ejecución, no a desarrollo.

Dockerfile multi-stage para Angular

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:alpine AS runtime

COPY --from=build /app/dist/ /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Idea

Angular se construye como archivos estáticos. La etapa final puede usar un servidor web liviano como Nginx.

Angular vs NestJS en Docker

Aspecto	Angular	NestJS
Tipo de aplicación	Frontend estático después del build	Backend Node.js en ejecución
Build	Genera archivos en dist/	Compila TypeScript a JavaScript
Runtime	Nginx u otro servidor estático	Node.js ejecutando dist/main.js
Puerto típico	80 dentro del contenedor	3000 dentro del contenedor

Relación con la arquitectura

Docker no cambia las responsabilidades: Angular sigue siendo frontend y NestJS backend.

Nombrar y etiquetar imágenes

Tag de imagen

Una etiqueta permite identificar una versión o variante.

```
docker build -t productos-backend:1.0 .  
docker build -t productos-backend:latest .
```

Convención habitual

```
usuario/repositorio:tag
```

```
sandracano/productos-backend:1.0  
ghcr.io/sandracano/productos-backend:1.0
```

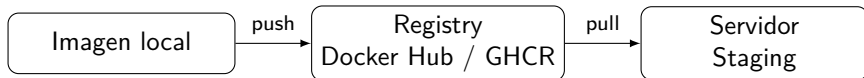
Idea

El tag debe permitir saber qué versión de la aplicación representa la imagen.

¿Qué es un registro de imágenes?

Registry

Es un servicio donde se almacenan, versionan y comparten imágenes de contenedores.



Ejemplos

Docker Hub, GitHub Container Registry, GitLab Container Registry, Amazon ECR, Azure Container Registry.

Publicar una imagen

Autenticarse

```
docker login
```

Construir y etiquetar

```
docker build -t usuario/productos-backend:1.0 .
```

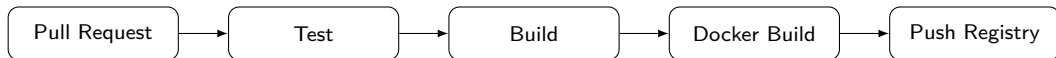
Subir al registry

```
docker push usuario/productos-backend:1.0
```

Descargar en otro ambiente

```
docker pull usuario/productos-backend:1.0
```

Del CI al registro de imágenes



Evolución del pipeline

Primero validamos el código. Luego podemos extender el pipeline para construir una imagen y publicarla en un registro.

Importante

No se debería publicar una imagen si fallan las pruebas, el build o una validación crítica.

Ejemplo de extensión en GitHub Actions

```
jobs:
  backend:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: backend

    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: backend/package-lock.json

      - run: npm ci
      - run: npm test
      - run: npm run build

      - name: Construir imagen Docker
```

Buenas prácticas iniciales

- Usar `package-lock.json` y `npm ci`.
- Copiar primero archivos de dependencias y luego el código.
- Usar `.dockerignore`.
- No copiar archivos `.env` reales ni secretos.
- Preferir multi-stage para imágenes de producción.
- Etiquetar imágenes con versiones o identificadores claros.
- Documentar cómo construir y ejecutar la imagen.
- Mantener consistencia entre comandos locales y pipeline CI.

Idea clave

Una imagen Docker debe ser reproducible, entendible y segura para el contexto donde será ejecutada.

Error frecuente: confundir arquitectura con contenedores

Docker no define por sí solo una arquitectura

Docker permite empaquetar componentes, pero no obliga a usar microservicios ni arquitectura orientada a eventos.

Decisión	Pregunta que responde
Docker	¿Cómo empaqueto y ejecuto el componente?
REST	¿Cómo se comunican cliente y backend?
Microservicios	¿Cómo divido responsabilidades del sistema?
Event-driven	¿Cómo reaccionan componentes ante eventos?

Para esta unidad

Primero aprenderemos a contenerizar componentes individuales. Luego podremos integrarlos con

Actividad 1: clasificar conceptos

Instrucción

Clasifique cada elemento como imagen, contenedor, Dockerfile, comando o registry.

Elemento	Clasificación
node:20-alpine	
docker run -p 3000:3000 ...	
productos-backend:1.0	
FROM node:20-alpine	
Docker Hub	
Contenedor ejecutando NestJS	

Solución: clasificar conceptos

Elemento	Clasificación
<code>node:20-alpine</code>	Imagen base
<code>docker run -p 3000:3000 ...</code>	Comando
<code>productos-backend:1.0</code>	Imagen etiquetada
<code>FROM node:20-alpine</code>	Instrucción de Dockerfile
Docker Hub	Registry
Contenedor ejecutando NestJS	Contenedor

Idea

La terminología correcta ayuda a diagnosticar errores y a comunicar decisiones técnicas.

Actividad 2: diseño de Dockerfile

Caso

El equipo tiene un backend NestJS que expone:

GET /productos

Preguntas

- 1 ¿Qué imagen base usaría?
- 2 ¿Qué archivos copiaría primero?
- 3 ¿Dónde ejecutaría `npm ci`?
- 4 ¿Qué puerto expondría?
- 5 ¿Cuál sería el comando final del contenedor?
- 6 ¿Qué agregaría al archivo `.dockerignore`?

Responda brevemente:

- 1 ¿Cuál es la diferencia entre imagen y contenedor?
- 2 ¿Para qué sirve un Dockerfile?
- 3 ¿Qué significa el punto final en `docker build -t app .`?
- 4 ¿Por qué conviene usar multi-stage build?
- 5 ¿Qué es un registro de imágenes?
- 6 ¿Qué archivo usaría para evitar copiar `node_modules` o `.env`?

Cinco ideas clave

- ➊ Una imagen es un paquete inmutable para crear contenedores.
- ➋ Un contenedor es una instancia en ejecución de una imagen.
- ➌ Un Dockerfile define cómo construir una imagen.
- ➍ Multi-stage separa construcción y ejecución.
- ➎ Un registry permite almacenar, versionar y compartir imágenes.

Conexión con la próxima clase

Después de contenerizar componentes individuales, el siguiente paso será ejecutar varios servicios juntos mediante Docker Compose.

- Docker Docs. *What is Docker?*
- Docker Docs. *What is an image?*
- Docker Docs. *Dockerfile overview.*
- Docker Docs. *Dockerfile reference.*
- Docker Docs. *Multi-stage builds.*
- Docker Docs. *What is a registry?*
- Docker Docs. *Docker Hub repositories.*